STAT214 Notes

1 Chapter 1: An Introduction to Veridical Data Science

1.1 The PCS Framework

- **Predictability**: Your conclusions are considered to be predictable if you can show that they reemerge in relevant future data and/or are in line with existing domain knowledge.
- **Computability**: You typically don't need to demonstrate that your results are computable; rather you will use computation to conduct and evaluate your analyses, as well as to create data-inspired simulations.
- **Stability**: Your conclusions are considered stable if you can show that they remain fairly consistent when realistic perturbations (changes/modifications) are made to the underlying data, judgment calls, and algorithmic choices.

1.2 The Training, Validation, and Test Datasets

- **training set**: at least 50 percent of the original data; used to conduct your explorations, analyses, and to train your algorithms
- **validation set**: typically use 20 percent to 40 percent of the original data; used to demonstrate that your results reemerge
- **test set**: typically 20 percent of the original data; used to provide a final independent assessment of an algorithm

1.3 Dataset Splitting

- **Time-based split**: If your data was collected over time and you will be applying an algorithm that you have trained to future data
- **Group-based split**: If your data has a natural grouping, and you will be applying an algorithm that you train to a new group of data points
- **Random split**: If the current data points are all more or less exchangeable with one another

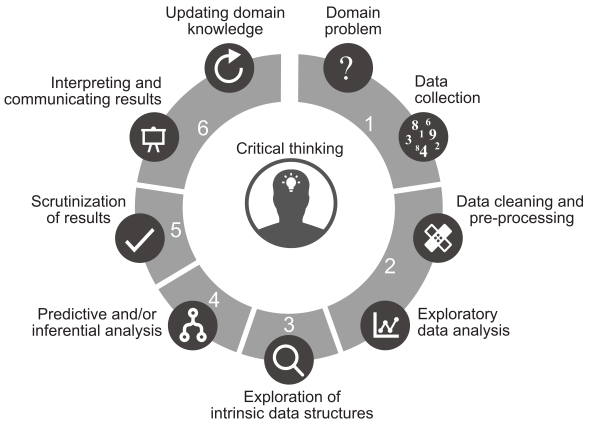
1.4 Investigate the Uncertainty

- recompute the results using slightly **different perturbed versions of the data**
- compare the results that are computed using **alternative versions of the data that have been cleaned and preprocessed**
- evaluate the results using **future data**

2 Chapter 2: The Data Science Life Cycle

2.1 The Data Science Life Cycle

1. Domain problem formulation and data collection
   - Formulating a Question
   - Data Collection
   - Making a Plan for Evaluating Predictability
2. Data cleaning, preprocessing, and exploratory data analysis
   - Data Cleaning
   - Preprocessing
   - Exploratory Data Analysis
3. Exploration of intrinsic data structures (dimensionality reduction and clustering)
   - Dimensionality reduction analysis
   - Cluster analysis
4. Predictive and/or inferential analysis
   - Prediction
   - Data-Driven Inference
5. Evaluation of results
6. Communication of results



3 Chapter 3: Setting Up Your Data Science Project

3.1 An Example File Structure (Using R)

```
data/
  data_documentation/
    codebook.txt
  data.csv
dslc_documentation/
  01_cleaning.qmd
  02_eda.qmd
  03_clustering.qmd
  04_prediction.qmd
  functions/
    cleanData.R
    preProcessData.R
README.md
```

- **`data/`**
  - raw dataset
  - data documentation information
- **`dslc_documentation/`**
  - separate `.qmd` Quarto file (for R) or `.ipynb` Jupyter Notebook (for Python) for conducting and documenting the code-based explorations and analysis at each stage of the DSLC (with numeric prefix)
  - `functions/` subfolder with the `.R` (R) or `.py` (Python) scripts
- **`README.md`**

3.2 Tips and Tricks for Reproducibility

- Avoid handling raw data or doing manual modifications
- Regularly rerun your code from start to finish in a fresh R/Python session
- Tidy up your code files
- Use version control (e.g., Git)
- Document everything
- Use up-to-date software

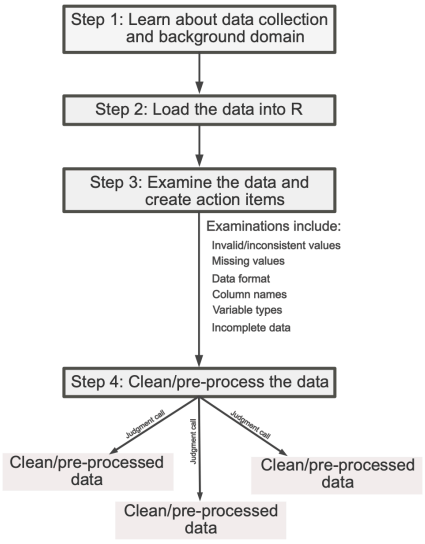
4 Chapter 4: Data Preparation

4.1 Overall Process of Data Preparation

1. **cleaning** your data by modifying it so it is appropriately formatted and unambiguous
2. **preprocessing** your data so it conforms to the formatting requirements required by whatever computational algorithms you plan to use to answer your question

4.2 Four-Step Data Cleaning Procedure

- Step 1: Learn about the data collection process and the problem domain
- Step 2: Load the data
- Step 3: Examine the data and create action items
- Step 4: Clean the data



4.2.1 Step 1: Learn About the Data Collection Process and the Problem Domain

Learning About the Domain Area

- What does each variable measure?
- How was the data collected?
- What are the observational units?
- Is the data relevant to my project?
- What questions do I have, and what assumptions am I making?

4.2.2 Step 2: Load the Data

Identify How to Load Data:

- How your data is being stored?
- If your data contains multiple tables, is there a key variable that connects them?
- What parts of the data are relevant to the question being asked?

Checking That Data Has Been Loaded Correctly:

- Look at the first and last 10 rows of your loaded data and compare it to the first 10 rows of the raw data file to confirm that they match
- Check that the dimension of your data matches what you expect

4.2.3 Step 3: Examine the Data and Create Action Items

Messy Data Trait 1: Invalid or Inconsistent Values

Suggested Examination:

- Look at randomly selected rows in the data, as well as sequential rows
- Print out the **minimum, maximum, and average values** of each numeric column
- Look at **histograms** of the distributions of each numeric variable
- Print out the **unique values** of categorical variables

Possible Action Items:

- Leave them as they are
- Replace the invalid/inconsistent values with the “correct” value
- Replace invalid values with missing values
- Convert measurements with different units to a common unit

Messy Data Trait 2: Improperly Formatted Missing Values

Suggested Examination:

- Print out the **minimum and maximum** of each numeric column
- Print out the **unique values** of categorical variables
- Look at **histograms** of the distributions of each numeric variable
- Print the **number (and the proportion) of missing values** in each column
- **Visualize the patterns of missing data** (e.g., using a heatmap)

Possible Action Items:

- Replace ambiguous missing values (e.g., `99999` or `"missing"`) with explicit missing values (`NA`)
- Replace them with "right" values and justify your choice

> ⚠ **Remark: A Note on Imputing Missing Values**
>
> Unless it is absolutely necessary (or your data contains a lot of observations—at least in the tens of thousands), we don’t recommend imputing variables that have more than half of their values missing.

Messy Data Trait 3: Nonstandard Data Format

Suggested Examination:

- What are the observational units? Is there a single row for each observational unit?
- Does each column correspond to a single type of measurement?

Possible Action Items:

- Pivot your data to a wider format if it is in a long format, or to a longer format if it is in an overly wide format
- Aggregate multiple measurements for individual observational units
- Separate columns that contain merged measurements

Messy Data Trait 4: Messy Column Names

Suggested Examination:

- Simply print the column names

Possible Action Items:

Modify column names to follow the style guide:

- lowercase
- human-readable
- words should be separated by underscores

Messy Data Trait 5: Improper Variable Types

Suggested Examination:

- Print the type of each column
- Look at individual values in each column

Possible Action Items:

- Converting each variable to the most appropriate type
- Reformatting the levels of categorical variables
- Aggregating uncommon/rare levels of a categorical variable

Messy Data Trait 6: Incomplete Data

Suggested Examination:

- Check that every observational unit appears in the data **exactly once**
- Check that the number of rows in the data matches what you expect, such as from the data documentation

Possible Action Items:

- If any observational units are missing from the data, add them to the data and populate unknown entries with missing values

4.2.4 Step 4: Clean the Data

Do not recommend saving a copy of your clean datasets to load for your analysis

4.2.5 Additional Common Preprocessing Steps

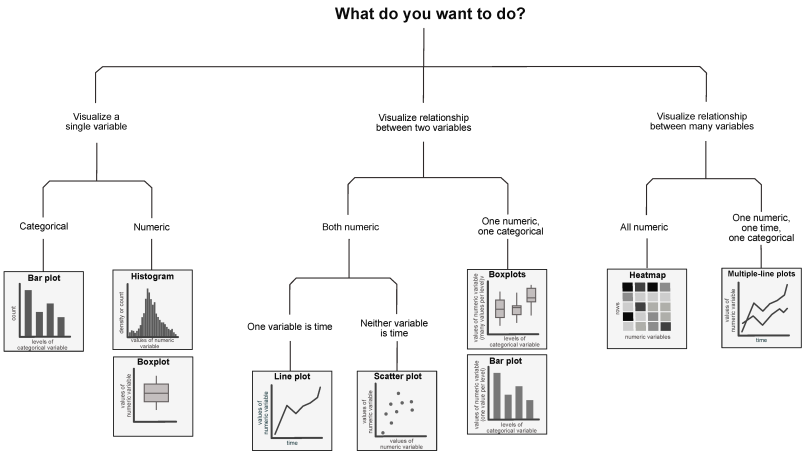
- Standardizing or normalizing variables that are on different scales
- Transforming variables so they follow a more Gaussian
- Splitting categorical variables into binary variables, called “one-hot encoding” in ML
- Featurization

5 Chapter 5 Exploratory Data Analysis

5.1 Choosing a Data Visualization Technique

Decide which visualization technique to use:

- based on **the types of variables** that are involved in your exploratory question
- by investigating the **stability** of your data visualization judgment calls



5.2 Tips for Explanatory Data Visualizations

Some tips for creating effective presentation-quality polished explanatory data visualizations include:

- Ensuring that the plot has a **clear takeaway message**
- Use **color** thoughtfully and sparingly
- Use **size, color, and text** to guide the audience's attention and highlight a particular element of the chart
- Use **transparency** to reduce "overplotting"
- **Titles, axes, and legend text** should be meaningful and human-readable, and legend elements should be ordered in a meaningful way.
- investigate the effect of these data visualization judgment calls in a **stability analysis**

5.3 Common Exploration

- Mean, Median, and Quantile
- Histograms and Boxplots
- Variance and Standard Deviation
- Covariance and Correlation
  - Scatterplots
  - Log Scales
  - Correlation Heatmaps

5.4 PCS Scrutinization of Exploratory Results

- ***Predictability***
  - find external data and verify
  - conduct a literature search
- ***Stability***
  - **Stability to Data Perturbations**
    - perturb the data by adding some "noise"
  - **Stability to Cleaning and Preprocessing Judgment Call Perturbations**
    - identify any judgment calls that we made during these stages that might have influenced our results
    - conduct some stability assessments to try to visualize the uncertainty arising from these judgment calls

- **Stability to Data Visualization Judgment Call Perturbations**
  - investigate whether our findings are stable to alternative plausible visualization judgment calls

6 Chapter 6 Principal Component Analysis

6.1 Preprocessing: Standardization for Comparability

- **Standardization**
  - mean-centering
  - SD-scaling
- **Comparability**
  - There will be some situations in which the relative scale of the variables is meaningful, and scaling them will remove this meaning

6.2 The Proportion of Variability Explained

An important result is that **the sum of the variance of the columns in the original dataset equals the sum of the squared singular values**:

$$\sum_j \text{Var}[X_j] = \sum_j d_j^2$$

the proportion of the variability of the original data that is captured or explained by the j th principal component is given by

$$\text{prop of variability explained by PC}_j = \frac{d_j^2}{\sum_i d_i^2}$$

6.3 Scree Plot

The **scree plot**:

- is a common method for assessing how many principal components to keep in the reduced dimension dataset
- shows the proportion of variability explained by each principal component either in a bar chart or as a line plot.

6.4 Gaussianity Assumption

When applying principal component analysis,

- one common assumption is that the variables all follow a theoretical multivariate **Gaussian** (normal) distribution
- one way to check for Gaussianity formally is to use a **Q-Q plot** (“quantile-quantile” plot)

6.5 Transformation to Gaussian

- A common transformation for making skewed/asymmetrical data look more Gaussian is the **logarithmic transformation**
- make a judgment call based on whether a log-transformation improves the symmetry of each variable, and later evaluate the impacts of this decision in a PCS stability analysis

6.6 Principal Component Analysis Step-by-Step Summary

1. *(Preprocessing)* Consider whether it makes sense to **mean-center** and/or **SD-scale** the variables
2. *(Preprocessing)* Visualize the distribution of each variable to determine whether a **log-transformation** may increase the symmetry of the distributions.
3. Apply **SVD** to your numeric data matrix to conduct principal component analysis
4. Based on the right-singular vector matrix, V, identify which variables are contributing most to each principal component. Based on the diagonal singular value matrix, D, compute the **proportion of variability explained by each principal component**.
5. Compute the principal component transformation of the original data matrix, X, by multiplying the original data by the right-singular vector matrix, V, as in $X^{PC} = XV$.

6.7 PCS Evaluation of Principal Component Analysis

- ***Predictability***
  - Recompute the summary variables using the **external dataset**, and check that these new principal components are similar to those that were computed.

- ***Stability***
  - **Stability to Data Perturbations**
    - sampling uncertainty: using bootstrap data perturbation technique
    - measurement uncertainty
  - **Stability to Cleaning and Preprocessing Judgment Calls**

6.8 Alternatives to Principal Component Analysis

Common alternative approaches include:

- nonnegative matrix factorization (Lee and Seung 1999)
- independent component analysis (Hyvärinen, Karhunen, and Oja 2001)
- sparse dictionary learning (sparse coding) (Olshausen and Field 1997)
- kernel principal component analysis (Schölkopf, Smola, and Müller 1998)
- UMAP (McInnes, Healy, and Melville 2020)
- t-SNE (Van der Maaten and Hinton 2008)
- autoencoder (Goodfellow, Bengio, and Courville 2016).

---

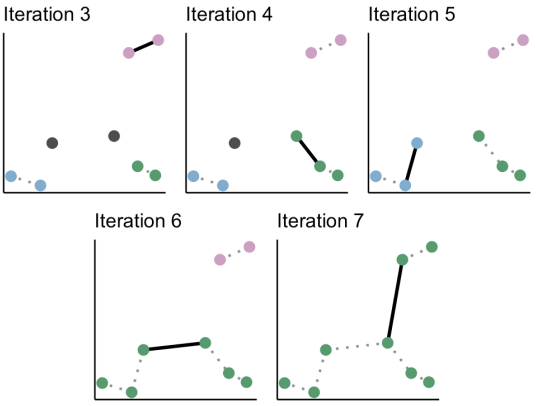
7 Chapter 7 Clustering

7.1 Hierarchical Clustering

7.1.1 Hierarchical Clustering

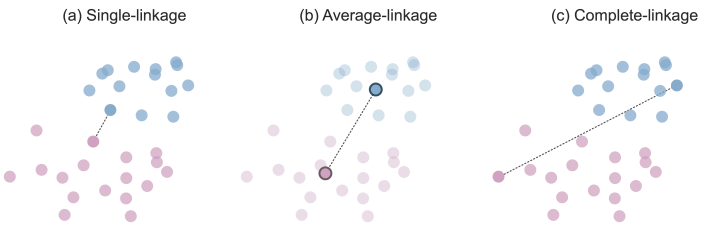
The hierarchical clustering algorithm

- starts by placing every observation in its own cluster
- then **iteratively merges the two most similar clusters** based on a cluster similarity measure.



7.1.2 Defining Intercluster Similarity (Linkage Methods)

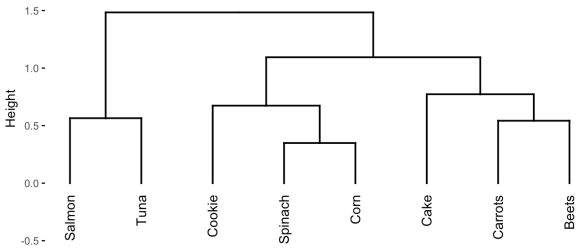
- **Single-linkage hierarchical clustering**: based on the similarity between the two most similar points from each cluster
- **Average-linkage hierarchical clustering**: based on the similarity between the two average points from each cluster
- **Complete-linkage hierarchical clustering**: based on the similarity between the two most dissimilar points from each cluster
- **Ward's linkage hierarchical clustering**: merging the two clusters whose merger will minimize the total within-cluster variance, which is defined as the sum of the variances of the data points within each cluster



7.1.3 Dendrogram

To help identify some interesting intermediate clusters we can generate a hierarchical clustering dendrogram

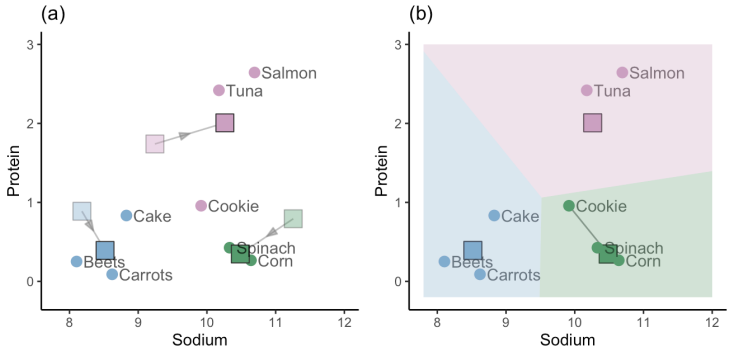
A **dendrogram** is a visual representation of the sequence of cluster merges (linkages).



7.2 K-Means Clustering

7.2.1 K-Means Clustering

1. Randomly place initial cluster centers within the data space.
2. Assign the cluster membership of each data point based on the initial cluster center it is most similar/closest to.
3. Calculate new cluster centers for each cluster based on the average of all the points that were assigned to the corresponding cluster.
4. Redefine the cluster membership of each point based on the new cluster centers.
5. Repeat steps 3 and 4 until the clusters stop changing (i.e., until the algorithm has converged).



7.2.2 Preprocessing: Transformation

- Although neither the K-means nor the hierarchical clustering algorithms require Gaussian or symmetric variables, **transforming skewed variables (e.g., using a log transformation)** can often lead to more meaningful clusters if the transformation improves the symmetry of the variable distribution.
- mean-centering has no impact on the clusters

7.3 Visualizing Clusters in High Dimensions

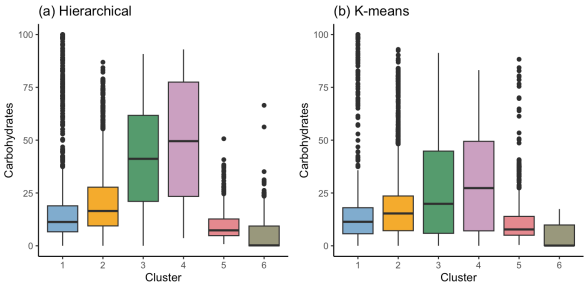
7.3.1 Sampling Data Points from Each Cluster

One way to visualize a set of clusters is to look at a sample of the data points that ended up in each cluster.

1	2	3	4	5	6
Strawberries	Chicken	Veal	Cheese	Collards	Sardines
Jelly	Tuna	Mayonnaise	Cornmeal	Squash	Octopus
Energy	Chocolate	Spinach	Cheese	Spinach	Herring
Custard	Egg	Tortilla	Graham	Pasta	Fish
Jelly	Bread	Caramel	Coffee	Broccoli	Octopus
Fruit	Bread	Pastry	Cheese	Steak	Salmon
Tea	Salisbury	Cereal	Yogurt	Mustard	Salmon
Prunes	Corn	Frankfurter	Ice	Carrots	Roe
Coffee	Spanish	Seasoned	Ice	Okra	Porgy
Jagerbomb	Strawberry	Egg	Ice	Artichoke	Herring
Gelatin	Bread	Kung	Cheese	Carrots	Salmon
Plum	Tuna	Cheeseburger	Fudge	Cauliflower	Ray
Apple	Infant	Hamburger	Honey	Corn	Sardines
Cherries	Chocolate	Frankfurter	Coffee	Fruit	Trout
Soup	Infant	Beef	Cheese	Rice	Porgy

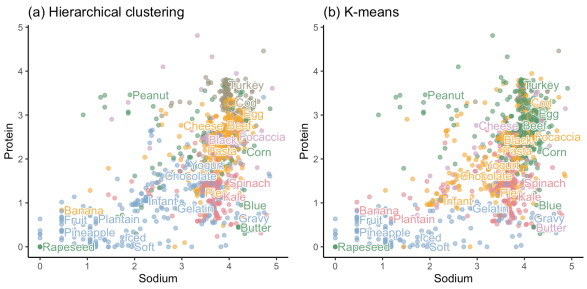
7.3.2 Comparing Variable Distributions across Each Cluster

Another way to visualize the clusters is to visualize the distribution of the individual variables for the observations in each cluster.



7.3.3 Projecting Clusters onto a Two-Dimensional Scatterplot

we can visualize them in a two-dimensional scatterplot by using the cluster membership to color each data point.



7.4 Quantitative Measures of Cluster Quality

7.4.1 Within-Cluster Sum of Squares (WSS)

The WSS for a set of clusters quantifies cluster tightness by adding together the squared Euclidean distances from each observation to its cluster center.

A general mathematical formula for computing the WSS for a set of K clusters in a dataset with p variables is given by

$$WSS = \sum_{k=1}^{K} \sum_{i \in C} \sum_{j=1}^{p} (x_{i,j} - c_{k,j})^2$$

> ⚠ **Remark: Scale WSS**
>
> The WSS can be made comparable across datasets with different numbers of observations by scaling the WSS by the number of observations involved in its computation.

7.4.2 The Silhouette Score

Define:

- a_i: The average distance between the th data point and the other points in the *same* cluster as i.
- b_i: The average distance between the th data point and the points in the *nearest adjacent* cluster.

Then the Silhouette scores is defined as

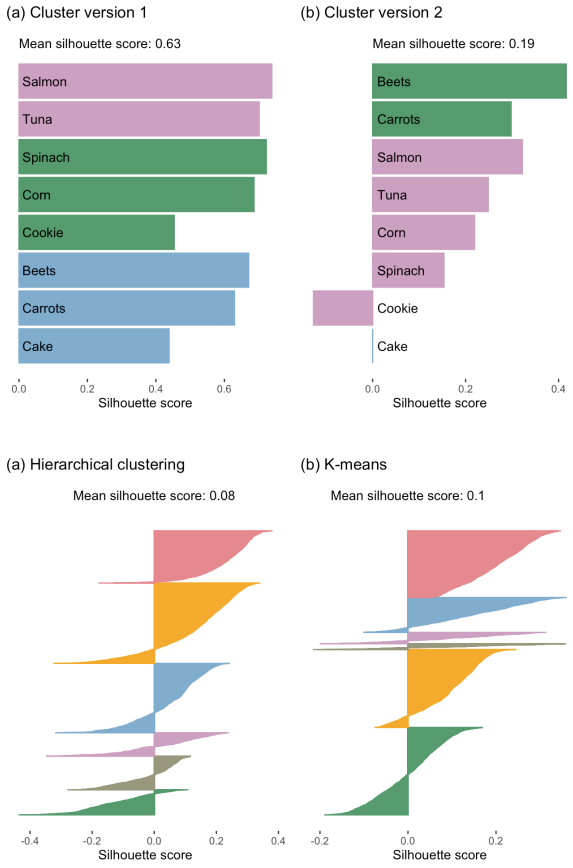
$$\text{silhouette score}_i = \frac{b_i - a_i}{\max(a_i, b_i)}$$

A data point that has a silhouette score of approximately:

- 1 is well clustered since it is much closer to the data points in its own cluster than it is to data points in other clusters.
- 0 lies between clusters since it is equally close to the data points in its own cluster and in another cluster.
- -1 is poorly clustered since it is further from the points in its own cluster than it is to points in another cluster.

7.4.3 The Silhouette Plot

Silhouette scores are often visualized using a **silhouette plot**, where the scores are plotted in descending order as horizontal bars (one for each observation) grouped by cluster.



7.5 The Rand Index for Comparing Cluster Similarity

7.5.1 Rand Index

The Rand Index is a measure that compares two possible ways of clustering a particular collection of data points in terms of how similarly the data points are clustered in each.

$$\text{Rand Index} = \frac{\#\text{ pairs in agreement}}{\#\text{ total pairs}}$$

7.5.2 The Adjusted Rand Index

An alternative to the Rand Index is the **adjusted Rand Index**, which

- lies between -1 and 1
- has been “corrected for chance”

$$\text{Adjust Rand Index} = \frac{\text{Rand Index} - \text{expected Rand Index}}{\max(\text{Rand Index}) - \text{expected Rand Index}}$$

7.6 Choosing the Number of Clusters

7.6.1 Using Cross-Validation to Choose K

V -fold CV aims to emulate the training/validation split for evaluating results on "pseudo"-validation data. The general process is as follows:

1. Split the data into V equal-sized non-overlapping subsets, called *folds*.
2. Remove the first fold (this fold will play the role of the pseudo-validation set), and use the remaining $V-1$ folds (the pseudo-training set) to train the algorithm.
3. Use the withheld first fold (the pseudo-validation set) to evaluate the algorithm that you just trained on the other $V-1$ folds using a relevant performance measure.
4. Replace the first fold, and remove the second fold (the second fold is now the pseudo-validation set). Train the algorithm using the other $V-1$ folds (including the previously withheld first fold). Evaluate the algorithm on the withheld second fold.
5. Repeat step 4 until each fold has been used as the pseudo-validation set, and you have V values of the performance measure for your algorithm.
6. Combine (e.g., compute the mean of) the V performance measure values, each computed on a withheld fold.

The split into training and validation sets (and test set, where relevant) should be informed by how the results will be applied to *future data* (e.g., using a time-based, group-based, or random split).

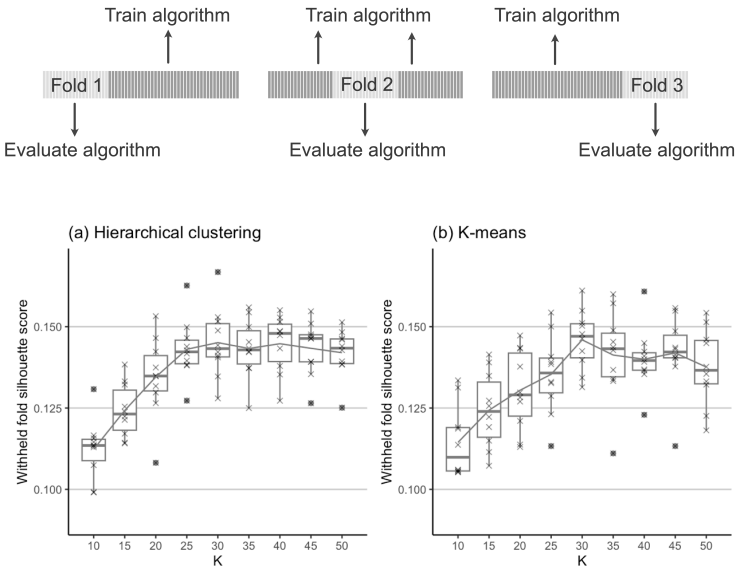


Figure 7.24: Boxplots demonstrating the distribution of CV silhouette scores for values of K (number of clusters) ranging from 10 to 50 by intervals of 5 for (a) the hierarchical clustering algorithm (with Ward linkage), and (b) the K-means algorithm. A line displaying the mean of the average silhouette scores for each K is shown, and each individual withheld-fold silhouette score is shown as an "$times$".

Remark

While CV provides a great lightweight PCS analysis for hyperparameter selection, we recommend that you still perform a detailed PCS evaluation of your final results.

7.6.2 Number of Folds

- 5-fold CV (i.e., $V=5$) is common for small datasets (i.e., datasets with up to a few thousand data points)
- 10-fold ($V=10$) CV is common for larger datasets.
- The extreme case where you let the number of folds, V, be equal to the number of observations (i.e., each observation is its own fold) is called **leave-one-out cross-validation**.

7.7 PCS Scrutinization of Cluster Results

7.7.1 Predictability

Evaluate the predictability by considering how well an **external dataset** get clustered when their cluster membership is determined based on which of the original cluster centers it is closest to.

- compare the WSS (we may need to divide the WSS by the number items in dataset)
- see whether some randomly chosen newly clustered items match the categories that we identified before.

7.7.2 Stability

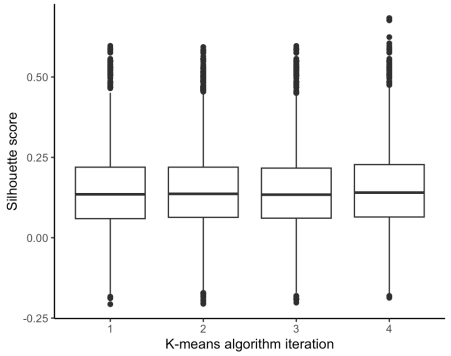
7.7.2.1 Stability to the Choice of Algorithm

Compare using the K-means algorithm versus the hierarchical clustering algorithm:

- average silhouette score (distribution of the silhouette score)
- (Adjusted) Rand Index
- whether the clusters identified are similar

7.7.2.2 Stability to Algorithmic Randomness

- compared the distribution of the silhouette score
- (Adjusted) Rand Index
- whether the clusters identified are similar



7.7.2.3 Stability to Data Perturbations

Create perturbed versions of the data, where the perturbations each correspond to

- a bootstrap sample of the original dataset
- adding a small amount of random noise to the nutrient measurements

then compare

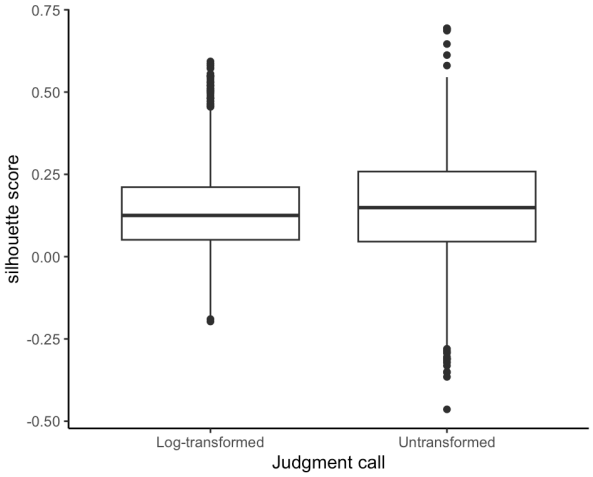
- average silhouette score (distribution of the silhouette score)
- (Adjusted) Rand Index

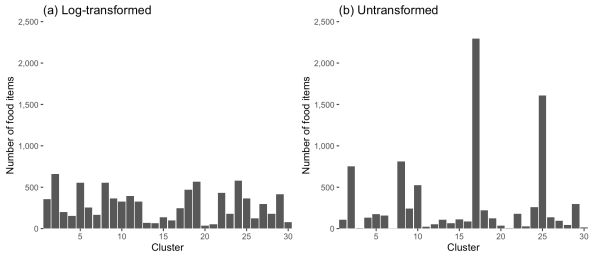
7.7.2.4 Stability to Preprocessing Judgment Call Perturbations

Since SD-scaling is necessary to ensure that variables with larger values do not dominate the distance calculations, we will focus on exploring how our results change when we log-transform *all* the variables versus *none* of the variables.

then compare

- average silhouette score (distribution of the silhouette score)
- number of items in each cluster


0.75
0.50
0.25
0.00
-0.25
-0.50
silhouette score
Log-transformed
Untransformed
Judgment call


(a) Log-transformed
(b) Untransformed
Number of food items
Cluster
2,500
2,000
1,500
1,000
500
0
5
10
15
20
25
30